

Question No:9

Question description

On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the students.

So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue.

there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

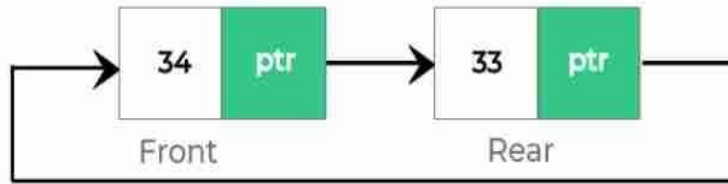
Being an exciting youngster can you solve it?

Function Description

enqueue(34)



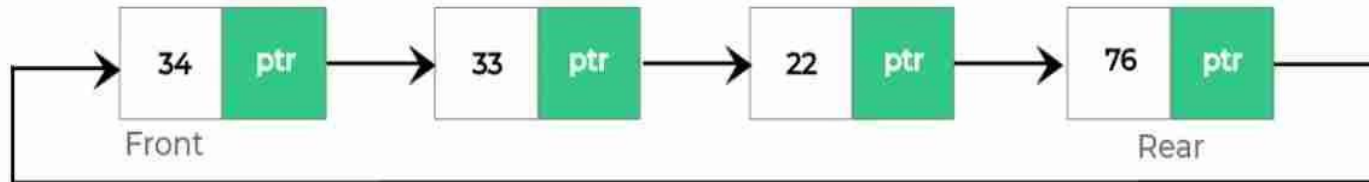
enqueue(33)



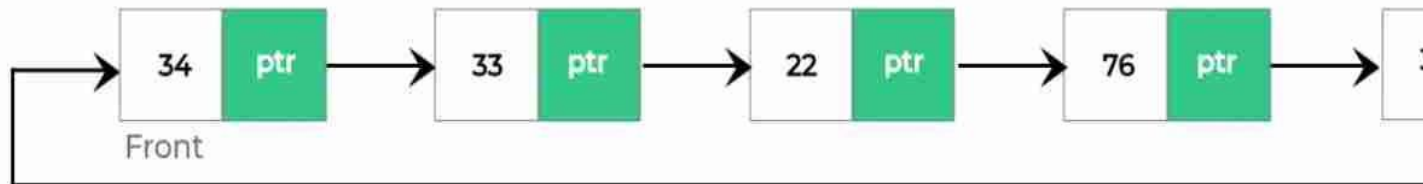
enqueue(22)



enqueue(76)

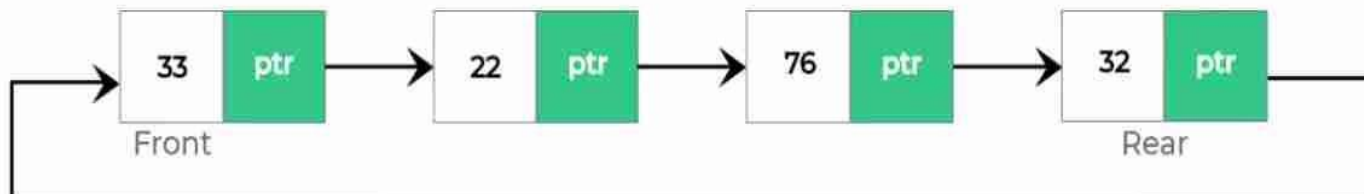


enqueue(32)



Removing the elements from Queue

dequeue()



Constraints:

0 < size < 100

0 < data < 1000

Input format:

First line indicates the size of the queue
Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept
second line indicates the one element dequeued from the queue using linked list concept.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
9 1 5 3 5

EXPECTED OUTPUT

9 1 5 3 5
1 5 3 5
5 3 5

Test Case 2

INPUT (STDIN)

8
1 8 6 4 9 5 3 5

EXPECTED OUTPUT

1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node
4  {
5      int data;
6      struct Node *next;
7  };
8  struct Node *front = NULL;
9  struct Node *rear = NULL;
10 void enqueue(int data)
11 {
12     struct Node *newNode;
13     newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = data;
15     newNode->next = NULL;
16     if (front == NULL)
17     {
18         front = newNode;
19         rear = newNode;
20     }
21     else
22     {
23         rear->next = newNode;
24         rear = newNode;
25     }
26 }
27 int dequeue()
28 {
29     struct Node *temp;
30     if (front == NULL)
```

```
30     int value;
31
32     if (front == NULL)
33     {
34         return -1;
35     }
36     temp = front;
37     value = temp->data;
38
39     front = front->next;
40
41     if (front == NULL)
42     {
43         rear = NULL;
44     }
45     free(temp);
46     return value;
47 }
48 void display()
49 {
50     struct Node *temp = front;
51
52     while (temp != NULL)
53     {
54         printf("%d", temp->data);
55
56         if (temp->next != NULL)
```

```
56         if (temp->next != NULL)
57         {
58             printf(" ");
59         }
60
61         temp = temp->next;
62     }
63     printf("\n");
64 }
65 int main()
66 {
67     int n;
68     int data;
69     scanf("%d", &n);
70     for (int i = 0; i < n; i++)
71     {
72         scanf("%d", &data);
73         enqueue(data);
74     }
75     /* Display original queue */
76     display();
77     /* Dequeue one element */
78     dequeue();
79     /* Display after first dequeue */
```

```
80     display();
81     /* Dequeue second element */
82     dequeue();
83     /* Display after second dequeue */
84     display();
85     return 0;
86 }
```

Output:

Test Case-1:

```
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
```

Test Case-2:

```
8
1 8 6 4 9 5 3 5
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
PS C:\Users\kesha\OneDrive\Documents\3sem\C>
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236